

ÜBUNG 5

24WS-UE05-A01 Primzahlen

Erstellen Sie folgende Funktionen, um für eine gegebene Zahl zu ermitteln, ob diese eine Primzahl bzw. Primzahl-Dublette ist. Eine Primzahl-Dublette besteht dabei aus zwei Primzahlen, deren Differenz gleich 2 ist (z.B. 3 und 5, oder 11 und 13).

- `public static boolean isPrimeNumber(int number)` ermittelt für eine ganze Zahl `number`, ob es sich um eine Primzahl handelt. Falls `number` eine Primzahl ist, soll die Funktion als Ergebnis `true` liefern und sonst `false`. Für Zahlen kleiner zwei, soll die Funktion `false` als Ergebnis liefern.
- `public static boolean hasPrimeDoublette(int number)` ermittelt für die übergebene Zahl `number`, ob es sich dabei um eine Primzahl-Dublette handelt (die Zahl selbst sowie eine andere Zahl mit Differenz 2 oder -2 zur gegebenen Zahl sind jeweils eine Primzahl). Rufen Sie dabei die Funktion `isPrimeNumber` von oben auf, um zu überprüfen, ob eine bestimmte Zahl eine Primzahl ist.

isPrimeNumber Funktion implementiert Keine Ergebnisse

hasPrimeDoublette Funktion implementiert Keine Ergebnisse

Beispiel main Methode

```
public static void main(String [] args) {  
    Out.println("***** prime numbers between 25 and 50 *****");  
    for (int i = 25; i<=50; i++) {  
        if (isPrimeNumber(i)) {  
            Out.print(i + " is prime");  
            if (hasPrimeDoublette(i)) {  
                Out.print(" + doublette");  
            }  
            Out.println();  
        }  
    }  
}
```

Beispiel Ausgabe

```
***** prime numbers between 25 and 50 *****  
29 is prime + doublette  
31 is prime + doublette
```

37 is prime

41 is prime + doublette

43 is prime + doublette

47 is prime

24WS-UE05-A02 Preisberechnung

Ein Supermarkt eröffnet einen Online-Shop. Es gibt unterschiedliche Varianten der Preisberechnung. Implementieren Sie die aufgelisteten Funktionen zur Berechnung des Gesamtpreises des Lebensmittel-Einkaufs durch Überladen der Funktionen:

- `public static double calculatePrice(int amount, double price)` berechnet den Gesamtpreis aus der Anzahl und dem Stückpreis. Runden Sie den Gesamtpreis auf 2 Kommastellen. Bei ungültigen Parameterwerten oder ungültigem Ergebnis soll -1 als Ergebnis geliefert werden.
- `public static double calculatePrice(int amount, double price, int discountInPercent)` berechnet den Gesamtpreis aus der Anzahl und dem Stückpreis und gewährt einen Rabatt in Prozent. Runden Sie den Gesamtpreis auf 2 Kommastellen. Bei ungültigen Parameterwerten oder ungültigem Ergebnis soll -1 als Ergebnis geliefert werden.
- `public static double calculatePrice(int amount, double price, double totalDiscount)` berechnet den Gesamtpreis aus der Anzahl und dem Stückpreis und gewährt einen fixen Rabatt (in €). Runden Sie den Gesamtpreis auf 2 Kommastellen. Bei ungültigen Parameterwerten oder ungültigem Ergebnis soll -1 als Ergebnis geliefert werden.

Hinweise:

- Der errechnete Gesamtpreis darf nicht kleiner 0 sein!
- Genauso darf auch kein negativer Rabatt gewährt werden.
- Für das Runden von Zahlen inkl. Nachkommastellen können Sie die Funktion `Math.round()` verwenden. Beispiel:

```
double x = 123.456
x = (double) Math.round(x * 100) / 100;
Out.println(x); //x hat jetzt den Wert 123.46
```

Funktion `calculatePrice(int, double)` implementieren Keine Ergebnisse

Funktion `calculatePrice(int, double, int)` implementieren Keine Ergebnisse

Funktion `calculatePrice(int, double, double)` implementieren Keine Ergebnisse

Lesen Sie in der `main`-Methode die entsprechenden Werte vom Benutzer ein und rufen Sie Ihre Funktionen mit diesen Werten auf. Alternativ können Sie auch automatisch testen, ohne Werte einzulesen.

24WS-UE05-A03 Windchill

Der Windchill (engl.) bzw. die Windkühle beschreibt den Unterschied zwischen der gemessenen Lufttemperatur und der gefühlten Temperatur in Abhängigkeit von der Windgeschwindigkeit. Er ist definiert für Temperaturen zwischen -273,15 und 10 Grad Celsius.

Im Umkehrschluss: Wenn Sie die Windchill-Temperatur (wct) und die tatsächliche Temperatur (t) kennen, können Sie sich daraus die aktuelle Windgeschwindigkeit (v) berechnen. Dafür gilt folgende Formel:

$$v = (wct - 13.12 - 0.6215 * t) / (0.3965 * t - 11.37) * 6.25$$

Erstellen Sie eine Funktion `public static double calculateWindSpeed(double temp, double windchill)`, welche die Windgeschwindigkeit auf Basis des Windchill und der tatsächlichen Lufttemperatur berechnet und als Ergebnis liefert. Bei ungültigen Eingaben soll -1 als Ergebnis geliefert werden.

Funktion `calculateWindSpeed(double, double)` implementieren Keine Ergebnisse

Erstellen Sie zusätzlich eine Funktion `public static double roundWindSpeed(double v, int decimalPlaces)`, welche eine Windgeschwindigkeit v auf die gewünschten Dezimalstellen `decimalPlaces` genau rundet und die gerundete Windgeschwindigkeit als Ergebnis liefert. Sind die Dezimalstellen oder die Windgeschwindigkeit negativ, soll -1 als Ergebnis geliefert werden.

Funktion `roundWindSpeed(double, int)` implementieren Keine Ergebnisse

Erstellen Sie außerdem eine Funktion `public static boolean checkTempForWindchill(double temp)` die für einen gegebenen Temperaturwert `temp` überprüft, ob für diesen ein Windchill berechenbar ist. Falls der Windchill für den angegebenen Temperaturwert berechenbar ist, soll die Funktion `true` zurückgeben, ansonsten `false`.

Funktion `checkTempForWindchill(double)` implementieren Keine Ergebnisse

Testen Sie alle drei Funktionen, indem Sie in der `main`-Methode die Lufttemperatur und die Windchill vom Benutzer einlesen und die Funktionen aufrufen.

24WS-UE05-A04 Dreiecke

Erstellen Sie Funktionen wie unten beschrieben, welche überprüfen, ob es sich bei einem Dreieck um ein rechtwinkeliges, ein gleichseitiges, bzw. ein gleichschenkeliges Dreieck handelt.

Zusätzlich zu den drei Seitenlängen, ist jeweils eine Genauigkeit `epsilon` gegeben, mit der die Gleichheit überprüft werden soll. Zwei `double` Werte sind also dann als gleich anzusehen, wenn die Differenz kleiner als `epsilon` ist. zB.:

```
double epsilon = 0.001;
double x = 1.0001;
double y = 1.0002;
if (Math.abs(x-y) < epsilon) {
    Out.println("x gleich y");
}
```

Erstellen Sie außerdem eine Funktion zur Validierung der Eingaben. Kann aus den gegebenen Parametern ein Dreieck gebildet werden? Siehe <https://www.wikihow.com/Determine-if-Three-Side-Lengths-Are-a-Triangle>

- `public static boolean isTriangle(double a, double b, double c)` überprüft, ob aus den angegebenen Parametern ein Dreieck gebildet werden kann. Dies ist der Fall, wenn 2 der drei übergebenen Seiten in Summe immer größer als die dritte Seite sind, also
 - $a + b > c$, UND
 - $a + c > b$, UND
 - $b + c > a$
- `public static boolean isRightAngled(double a, double b, double c, double epsilon)` überprüft ob es sich beim angegebenen Dreieck um ein rechtwinkeliges Dreieck handelt. Bei einem rechtwinkeligen Dreieck gilt: $a^2 + b^2 = c^2$
- `public static boolean isEquilateral(double a, double b, double c, double epsilon)` überprüft ob es sich beim angegebenen Dreieck um ein gleichseitiges Dreieck handelt. Bei einem gleichseitigen Dreieck gilt: $a = b = c$
- `public static boolean isIsosceles(double a, double b, double c, double epsilon)` überprüft ob es sich beim angegebenen Dreieck um ein gleichschenkeliges Dreieck handelt. Bei einem gleichschenkeligen Dreieck gilt: $a = b$ ODER $b = c$ ODER $a = c$.

Hinweis: Negative Seitenlängen ergeben kein gültiges Dreieck!

Erstellen Sie unter Verwendung der oben beschriebenen Funktionen eine zusätzliche Funktion `getTriangleKind(double a, double b, double c, double epsilon)`, welche für die übergebenen Parameter die Art des Dreiecks als vordefiniertes Zeichen zurückgibt, siehe unten.

- `public static char getTriangleKind(double a, double b, double c, double epsilon)` gibt den Typ des Dreiecks zurück.

- 'R' bei einem rechtwinkeligem Dreieck (verwenden Sie dafür die Methode `isRightAngled`)
- 'E' bei einem gleichseitigen Dreieck (verwenden Sie dafür die Methode `isEquilateral`)
- 'I' bei einem gleichschenkeligen Dreieck (verwenden Sie dafür die Methode `isIsosceles`)
- 'Z' bei einem Dreieck, das sowohl rechtwinkelig als auch gleichschenkelig ist
- 'T', wenn es sich um keine der bisher genannten Sonderformen handelt, aber aus den gegebenen Parametern ein Dreieck gebildet werden kann (verwenden Sie dafür die Funktion `isTriangle`)
- 'X', falls aus den gegebenen Parametern kein Dreieck gebildet werden kann (verwenden Sie dafür ebenfalls die Funktion `isTriangle`).

24WS-UE05-A05 Lustige Sieben

Schreiben Sie ein Java-Programm für das Würfelspiel lustige Sieben mit dem ein Spieler gegen die Bank spielen kann.

Der Spieler setzt einen Einsatz auf eine beliebige Zahl auf einer Tafel (siehe unterhalb). Der Bankier würfelt mit zwei Würfeln und zählt die Augen zusammen. Bei gleicher Zahl erhält der Spieler den doppelten und bei einer auf einer Längsseite der Spieltafel stehenden Zahl den einfachen Einsatz zurück. Ansonsten gewinnt die Bank. Fallen also zB. acht Augen, so bekommt man bei gesetzter Zahl acht den Einsatz doppelt, bei gesetzter Zahl 3, 5, 10 oder 12 den Einsatz einfach zurück, während man bei 7, 2, 4, 6, 9, 11 den Einsatz verliert. Wenn die Zahl 7 (Refait) gesetzt wurde und fällt, dann zahlt die Bank den Einsatz dreifach zurück.

7	
2	3
4	5
6	8
9	10
11	12

Lesen Sie den Einsatz des Spielers und die gesetzte Zahl ein. Wiederholen Sie die Eingabe bei ungültigen Werten. Nutzen Sie `Math.random()` für das Würfeln und bestimmen Sie nach jedem einzelnen Spiel das Ergebnis und geben Sie den Gesamtgewinn oder -verlust aus. Fragen Sie den Spieler anschließend, ob er weiterspielen möchte.

Beispieldialog:

Lass uns Spielen!

```
=====
=====
```

Bitte um Ihren Einsatz: 10

Einsatz: 10 EUR, Kontostand: -10 EUR

Setzen Sie auf eine Zahl zwischen 2 und 12: 5

Die Wuerfel sind gefallen: 5 + 3 = 8

Sie haben gewonnen und bekommen 10 EUR

Ihr neuer Kontostand lautet 0 EUR

Wollen Sie weiterspielen (j/n)? j

```
-----
```

Bitte um Ihren Einsatz: 10

Einsatz: 10 EUR, Kontostand: -10 EUR

Setzen Sie auf eine Zahl zwischen 2 und 12: 5


```
Die Wuerfel sind gefallen: 4 + 1 = 5
Sie haben gewonnen und bekommen 20 EUR
Ihr neuer Kontostand lautet 10 EUR
Wollen Sie weiterspielen (j/n)? j
-----
Bitte um Ihren Einsatz: 1000
Einsatz: 1000 EUR, Kontostand: -990 EUR
Setzen Sie auf eine Zahl zwischen 2 und 12: 5
Die Wuerfel sind gefallen: 4 + 3 = 7
Oje! Sie haben Ihren Einsatz in Hoehe von 1000 EUR verloren
Ihr neuer Kontostand lautet -990 EUR
-----
Ihr Gesamtgewinn/-verlust: -990 EUR
=====
```

Schreiben Sie mindestens die folgenden Funktionen bzw. Methoden und rufen Sie diese in der main-Methode auf, um einen Beispieldialog wie oben erzeugen zu können:

- `public static int rollDice()` würfelt mit einem Würfel und liefert eine zufällige Zahl von 1 bis 6
- `public static int readBid()` liest den Einsatz von der Nutzerin ein und liefert ihn als Ergebnis zurück. Der Einsatz wird dabei innerhalb der Methode vom Kontostand der Nutzerin abgezogen
- `public static int readBet()` liest die Zahl auf die der Benutzer setzt (von 2 bis 12) ein und liefert sie als Ergebnis zurück
- `public static int calculateWin(int dice, int bet, int bid)` berechnet den Gewinn auf Basis der geratenen Zahl, gewürfelten Zahl und gesetzten Summe. Die Funktion liefert -1 bei falschen Parameterwerten.
- `public static boolean playAgain()` fragt die Nutzerin, ob weitergespielt werden soll. Im Fall, dass weitergespielt werden soll, gibt die Methode `true` zurück, ansonsten `false`. Falls der Kontostand der Nutzerin bereits -100 € oder weniger beträgt, wird automatisch `false` zurückgegeben.
- `public static void playRound(int dice, int bet, int bid)` übernimmt unter Verwendung der Methode `calculateWin(int, int, int)` das Spielen einer Runde auf Basis der gegebenen Parameter. Dies umfasst neben dem Aufruf der Methode `calculateWin(int, int, int)` auch die Änderung des Kontostandes der Nutzerin auf Basis des Spielergebnisses, sowie die Ausgabe des Spielergebnisses auf der Kommandozeile.

Hinweis: um von mehreren Methoden aus auf den Kontostand zugreifen bzw. diesen verändern zu können, sollen Sie eine statische Variable `int total` verwenden.

24WS-UE05-A06 TicTacToe Connect-Three

Lesen Sie den gesamten nachfolgenden Text, bevor Sie mit Ihrer Lösung starten!

Schreiben sie ein Java-Programm, welches eine TicTacToe-Spiel Variation repräsentiert.

Die Variation ist angelehnt an das Spiel 4-Gewinnt/Connect-Four ([Siehe Link](#)).

Anstatt, dass ein*e Spieler*in willkürlich Zeichen auf dem TicTacToe-Spielfeld setzen kann, ist es nunmehr nur möglich, die Spalte auszuwählen und die Zeichen werden dann aufeinander gesetzt (siehe Beispiel in Konsolenausgabe).

Es soll ausgegeben werden, welche*r Spieler*in an der Reihe ist ("Player X" oder "Player O"). Anschließend soll die Spaltenauswahl von der*dem Spieler*in eingegeben werden. Diese geben an, wo der Kreis oder das Kreuz am Spielfeld platziert werden sollen.

Die eingegebene Ziffer gibt die Spalte an, in der das Zeichen aufgesetzt wird. Beispielsweise führt die Konsoleneingabe 3 dazu, dass das von unten erste freie Feld in der dritten Spalte befüllt wird.

Weiterhin soll die Konsoleneingabe der Spaltenauswahl einer Validierung unterzogen werden. Dabei sollen die folgenden Fälle überprüft werden:

- Die angegebene Spaltennummer darf nicht außerhalb des Spielfeldbereichs liegen.
- Bereits volle Spalten dürfen nicht mehr gewählt werden.

Das Spiel soll mit einer Erfolgsmeldung beendet werden, wenn ein*e Spieler*in drei gleiche Zeichen vertikal, horizontal oder diagonal platzieren konnte. Im Falle von Unentschieden (alle Felder befüllt allerdings kein*e Gewinner*in) soll ebenfalls eine entsprechend Ausgabe auf der Konsole ausgegeben werden.

Anstatt die gesamte Funktionalität in der main-Funktion zu deklarieren, sollen separate Funktionen erstellt werden. Nutzen Sie dafür die vorgefertigten Variablen und Methodenköpfe.

Ein typischer Spielverlauf könnte entsprechend wie folgt aussehen:

Player X: 2

+---+---+---+

| | |

+---+---+---+

| | |

+---+---+---+

| |X| |

+---+---+---+

Player O: 2

```
+---+---+---+
|  |  |  |
+---+---+---+
|  | O |  |
+---+---+---+
|  | X |  |
+---+---+---+
```

Player X: 2

```
+---+---+---+
|  | X |  |
+---+---+---+
|  | O |  |
+---+---+---+
|  | X |  |
+---+---+---+
```

Player O: 2

invalid move (column 2 is already full)

Player O: 4

invalid move (column 4 is outside the board)

Player O: 1

```
+---+---+---+
|  | X |  |
+---+---+---+
|  | O |  |
+---+---+---+
| O | X |  |
+---+---+---+
```

...

Player X: 3

```

+---+---+---+
| O | X | X |
+---+---+---+
| X | O | O |
+---+---+---+
| O | X | X |
+---+---+---+

```

This game ends in a draw!

Methodenköpfe und Variablen

```

public static final String DELIMITER_LINE = " +---+---+---+";
public static final String DELIMITER_SYMBOL = " | ";
public static final char EMPTY_CHAR = ' ';
public static final char X_PLAYER = 'X';
public static final char O_PLAYER = 'O';

// variables of the coordinates
public static char row1col1 = EMPTY_CHAR;
public static char row1col2 = EMPTY_CHAR;
public static char row1col3 = EMPTY_CHAR;
public static char row2col1 = EMPTY_CHAR;
public static char row2col2 = EMPTY_CHAR;
public static char row2col3 = EMPTY_CHAR;
public static char row3col1 = EMPTY_CHAR;
public static char row3col2 = EMPTY_CHAR;
public static char row3col3 = EMPTY_CHAR;

public static char currentPlayer = X_PLAYER;

public static void main(String[] args) {
    "Hier code einfügen"
}

```

```
/**
 * Places the current player character in the given column.
 */
public static void putInColumn(int userColumn) {
    "Hier code einfügen"
}

/**
 * Determines if the given column is already full on the game grid
 */
public static boolean isOccupied(int userColumn) {
    "Hier code einfügen"
}

/**
 * Determines whether the board grid is completely filled
 */
public static boolean boardIsFull() {
    "Hier code einfügen"
}

/**
 * Determines whether there is a winner on the board and returns the character of the winner.
 * In case there is no winner, function returns the EMPTY_CHAR.
 */
public static char getWinner() {
    "Hier code einfügen"
}

/**
 * Validates the input column number to be within the TicTacToe board range.
 */
public static boolean isOutsideBoard(int inputColumn) {
    "Hier code einfügen"
}
```

```
/**  
 * Prints out the TicTacToe board with all the values.  
 */  
public static void printBoard() {  
    "Hier code einfügen"  
}
```

ÜBUNG 6